FolderTreeExporter User Manual (English)

`FolderTreeExporter` is a Unity editor extension that generates, previews, and exports directory hierarchies within your Unity project as formatted plain text trees.
It is designed to streamline documentation workflows, such as creating README files for GitHub, BOOTH store descriptions, asset package specifications, and delivery structure diagrams.

Table of Contents

1. System Requirements
2. How to Launch
3. Window Layout & Basic Operations
4. Features & Output Examples
    - Output Modes (3 Modes)
    - Tree Styles (3 Styles)
    - Display Options
    - Copy & Export
5. Automatic Asset Type Classification
6. Recommended Use Cases & Settings

---

1. System Requirements

- **Supported Unity Versions**: Unity 2022.3.22f1 or higher, and Unity 6 (6000.x)
- **Supported OS**: Windows / macOS
- **Dependencies**: None (Operates using standard UnityEditor APIs)

---

2. How to Launch

You can open the tool window in either of the following two ways:

Method A: From the Top Menu Bar

From the Unity top menu bar, select `dennokoworks` > `FolderTreeExporter`.

Method B: From the Project Window Context Menu (Recommended)

Right-click any folder in the Project window and select `dennokoworks` > `Export Folder Tree`.
The window will open with the selected folder automatically assigned as the target.

---

3. Window Layout & Basic Operations

The window follows the dennokoworks floating design system and is composed of the following sections:

1. Top Bar (Header)

- **Window Title**: `FolderTreeExporter`
- **Version Label**: Displays the current local version (e.g., `v1.0.0`). The tool periodically checks the remote GitHub repository (`master` branch) in the background and highlights when an update is available in green.
- **Reload Button (↻)**: Instantly re-checks GitHub for newer releases.

2. Main Scrollable Area (Cards)

- **Target Folder Card**:
  - **Target Folder Field**: Displays the currently selected folder asset. You can also pick a folder directly using the Unity Object picker.
  - **"Browse" Button**: Opens a folder selection dialog to choose any folder under `Assets`.
  - **Drag & Drop Area**: Drag and drop any folder from the Project window into the box area to instantly set or switch the target.
- **Settings & Options Card**:
  - **Output Mode**: Select what elements to include in the tree (*Folders with Asset Types* / *Folders Only* / *With Files*).
  - **Tree Style**: Choose the formatting glyph style (*Standard Unicode Tree* / *ASCII Tree* / *Markdown List*).
  - **Show Asset Counts**: Toggles whether to display file counts next to each asset type tag (e.g., `[Texture: 5, Material: 2]`).
  - **Wrap in Code Block (`)**: Wraps the entire output with Markdown code fences (```` ``` ````).
- **Preview Card**:
  - Displays a real-time preview of the generated tree using a monospace font.
  - The text area is read-only, but supports mouse selection and select-all (`Ctrl+A` / `Cmd+A`) for manual copying.
  - The preview automatically updates whenever files or directories are added, deleted, or renamed within the project (you can also manually refresh via the "Refresh" button).

3. Footer (Action Buttons)

- **"Copy to Clipboard" Button**: Copies the entire generated tree text to your OS clipboard with a single click.
- **"Save to File" Button**: Opens a popup menu to save the output as either a plain text file (`.txt`) or a Markdown file (`.md`).

4. Status Bar

- Displays temporary confirmation messages upon copying or saving files.

> [!NOTE] For very large folder structures, the preview area truncates rendering after a safe threshold to prevent Unity rendering lag. However, the "Copy to Clipboard" and "Save to File" actions will always export the **complete, full-length** text.

---

4. Features & Output Examples

Output Modes (3 Modes)

1. Folders with Asset Types (Default)

Displays the directory structure along with tags indicating the types of assets contained directly within each folder (e.g., Model, Texture, Material). Highly recommended for package overviews and store pages.

```
MyAvatar
├── Animations [Animation]
├── Materials [Material]
├── Models [Model, Texture]
└── Prefabs [Prefab]
```

2. Folders Only

Outputs only the directory hierarchy without any file or asset details. Ideal when summarizing high-level project architecture.

```
MyAvatar
├── Animations
├── Materials
├── Models
└── Prefabs
```

3. With Files

Outputs the full tree including individual file names. Best suited for detailed delivery manifests and comprehensive file audits.

```
MyAvatar
├── Animations
│   └── Idle.anim
├── Materials
│   └── Body_Mat.mat
├── Models
│   ├── Body.fbx
│   └── Texture.png
└── Prefabs
    └── MyAvatar.prefab
```

Tree Styles (3 Styles)

Select between three formatting styles depending on where you plan to paste the output.

Style Name	Glyphs / Format	Recommended Destinations

Style Name	Glyphs / Format	Recommended Destinations
Standard Tree (Unicode) *(Default)*	`├──` , `└──` , `│`	GitHub, Discord, Notion, Code blocks
ASCII Tree	`+--` , `\--` , `|`	Monospace plain text, classic editors, legacy systems
Markdown List	`-` (2-space indent)	Proportional (variable-width) fonts, web forms, blogs

Comparison of Styles

- **Standard Tree (Unicode)**:

```
Folder
├── SubFolderA
│   └── AssetA
└── SubFolderB
```

- **ASCII Tree**:

```
Folder
+-- SubFolderA
|   \-- AssetA
\-- SubFolderB
```

- **Markdown List**:

```
- Folder
  - SubFolderA
    - AssetA
  - SubFolderB
```

Display Options

Show Asset Counts

When enabled in "Folders with Asset Types" mode, appends the count of files for each asset type.

- **OFF**: `Models [Model, Texture]`
- **ON**: `Models [Model: 1, Texture: 4]`

Wrap in Code Block (```)

When enabled, encloses the output in Markdown code block fences (```).

```
MyAvatar ├── Materials [Material: 3] └── Models [Model: 1, Texture: 4]
```

---

5. Automatic Asset Type Classification

Rather than relying purely on file extensions (such as `.png` or `.fbx`), `FolderTreeExporter` leverages Unity's `AssetDatabase` to classify assets by their true internal asset type.

- **Classification Examples**:
  - `Model`: 3D models (FBX, OBJ, VRM / glTF imports via ScriptedImporters)
  - `Prefab`: Unity Prefab assets
  - `Texture`: Textures and image assets (PNG, TGA, PSD, etc.)
  - `Material`: Materials
  - `Shader`: Shaders, ComputeShaders, ShaderIncludes, etc.
  - `Animation` / `Animator`: AnimationClips, AnimatorControllers
  - `Script` / `AssemblyDefinition`: C# scripts, asmdef files
  - `Scene`, `Audio`, `Video`, `Font`, `ScriptableObject`, `Text`, etc.
- **Exclusion Rules**:
  - `.meta` files are always automatically excluded.
  - Hidden files and directories starting with `.` (such as `.git`) are always excluded.

---

6. Recommended Use Cases & Settings

Use Case	Recommended Configuration
Discord / GitHub Issues & PRs	**Standard Tree** + **Wrap in Code Block: ON** → Guarantees perfect alignment in monospace font rendering.
BOOTH Product Description / Web pages	**Markdown List** OR **Standard Tree + Code Block** → Prevents horizontal misalignment in variable-width fonts.
Package README.md	**Folders with Asset Types** + **Standard Tree** + **Wrap in Code Block: ON** → Gives buyers and users an instant, clean overview of the package structure.
Delivery Manifests & File Audits	**With Files** + **ASCII Tree** or **Standard Tree**